

Cuprins

1.1	Obiective	1
1.2	Diagrame UML	2
1.2.1	Modele de proiectare	2
1.2.2	Noțiuni introductive UML	3
1.2.3	Tipuri de diagrame UML	5
1.3	Aplicații de desenare și modelare UML.....	15
1.3.1	Generalități	15
1.3.2	Comparații.....	16
1.3.3	Visual Paradigm for UML	19
1.3.4	Alte aplicații	19
1.4	Probleme propuse spre rezolvare / teme individuale.....	20

1.1 OBIECTIVE

- Prezentarea tipurilor de diagrame UML.
- Prezentarea modelelor de proiectare.
- Prezentarea și analiza principalelor aplicații de modelare/desenare UML.
- Utilizarea aplicației Visual Paradigm for UML pentru modelarea diagramelor UML.
- Realizarea diagramelor UML pentru proiectul ales.

1.2 DIAGrame UML

1.2.1 MODELE DE PROIECTARE

Un model de proiectare reprezintă o abstractizare a unei probleme și a soluției acesteia, care poate fi reutilizată în situații asemănătoare. Modelul reprezintă o descriere sau un șablon a unei metode de a rezolva o problemă (nu rezolvarea efectivă, concretă, a problemei). Acest șablon include de obicei cele mai bune practici care se doresc a fi folosite în rezolvarea unei clase de probleme (prezentând de exemplu relațiile și interacțiunile între clase și obiecte).

Reutilizarea modelelor de proiectare asigură un timp mai scurt de dezvoltare, creșterea fiabilității și flexibilității aplicației finale (aceste șabloane reprezintă soluții testate în timp, rafinate și validate cu fiecare implementare) și păstrarea unei linii generale comune în dezvoltarea diverselor aplicații.

Modelele de proiectare utilizează la bază module și relații între acestea. Ele pot fi clasificate (în funcție de criteriu) ca fiind:

- funcționale, structurale sau comportamentale (clasificarea uzuală);
- algoritmice, computaționale sau de implementare.

Ele sunt identificate printr-un nume și au atașate un set de descriptori care prezintă problema către care pot fi aplicate, soluția oferită și consecințele (pozitive și negative) ce rezultă din implementarea modelului. Șabloanele au caracter generic (nu depind de obicei de codul sursă sau de limbajul de implementare) și se reutilizează la nivelul proiectării sarcinii. O sarcină poate utiliza mai multe asemenea modele de proiectare, fiecare oferind soluția pentru o problemă punctuală.

La nivelul arhitecturii software se folosește modelul de proiectare arhitectural¹.

¹ Cum ar fi modelul „model-vizualizare-controller” (MVC – *model-view-controller*) care separă partea de logică a aplicației de partea de prezentare, permițând modificarea/testarea facilă a aplicației.

Dintre criticile utilizării excesive a șabloanelor de proiectare, cea mai importantă este că poate duce la dificultăți în a înțelege modelele complexe, complicându-le.

1.2.2 NOȚIUNI INTRODUCTIVE UML

Proiectele de mari dimensiuni sau care implică un număr mare de persoane în echipa de dezvoltare pot pune probleme cu privire la transmiterea eficientă și clară a informațiilor (mai ales când diferitele echipe nu sunt familiarizate cu uneltele și protocoalele utilizate de alte echipe). Pentru atingerea acestui scop este nevoie să se utilizeze o notatie standardizată, care să fie înțeleasă de toți membrii echipei.

Pentru a împărți o problemă complexă în subprobleme care pot fi „atacate” de echipe individuale, și pentru a obține un proiect final care să nu fie supradimensionat (sau din contra să nu aibă elemente lipsă) este necesară utilizarea unui model al proiectului, o simplificare care să rețină doar componentele critice ale acestuia (detaliile urmând a fi tratate individual). Pentru păstrarea consistenței trebuie folosit un formalism matematic sau un mod de reprezentare standardizat. Modelul poate fi utilizat static, pentru descrierea arhitecturii proiectului (din diverse puncte de vedere) dar și dinamic, funcțional (descriind relațiile între elementele modelului, evoluția sistemului în timp, etc.).

Limbajul de modelare UML (*Unified Modeling Language*)², are drept scop modelarea facilă și generică a modelelor. Reprezintă un standard (internațional) generic de notatie. Deși a fost creat inițial pentru a fi utilizat în ingineria software, el se poate utiliza acolo unde este nevoie de a exprima anumite modele într-un mod consistent, logic și formal. Pentru utilizarea efectivă a sa, este nevoie ca cerințele proiectului să fie definite într-o formă care poate fi înțeleasă și reprezentată formal.

² Apărut în 1997, standardul fiind administrat de OMG (Object Management Group).

Specificațiile UML 1.x s-au bazat pe utilizarea de tehnici de modelare a obiectelor (de exemplu dreptunghiuri pentru a reprezenta clase și obiecte). Specificațiile UML 2.x (acum ajunse la 2.4) cuprind patru părți³:

- superstructura (definește notațiile și semantica diagramelor și elementelor modelului);
- infrastructura (definește modelul de bază peste care este definită suprastructura);
- limbajul de descriere a constrângerilor de obiecte (definește regulile elementelor modelului);
- descrierea diagramelor UML (definește diferitele nivele ale diagramelor UML 2.x).

Componentele principale ale UML sunt descrise în continuare:

- Vederile. Acestea sunt abstractizări (organizatorice, funcționale, statice, etc.) ale sistemului, bazate pe diagrame.
- Diagramele. Structuri (grafuri) ce descriu conținutul vederilor. Există mai multe tipuri de diagrame, construite în funcție de aspectele particulare ce se doresc modelate.
- Elementele de modelare. Concepte folosite în modelare, cu corespondență în programarea OO (clase, obiecte, mesaje, etc). Reprezentarea acestora este aceeași în diferitele tipuri de diagrame.
- Mecanismele generale ce permit introducerea de informații și comentarii cu privire la un anumit element.

Standardul UML definește în număr de mecanisme comune care asigură integritatea notațiilor. Aceste mecanisme cuprind:

- stereotipurile (specializează clasele metamodelului și sunt de ajutor mai ales atunci când semantica elementului este insuficientă);
- etichetele (extind atributele claselor metamodelului, definind un nume și o valoare pentru fiecare proprietate a elementului modelat);

³ <http://www.omg.org/spec/UML/2.3/>

- notele (pot conține orice tip de informație – textul, fragmente de cod, referințe la alt document; sunt practic explicații atașate modelului);
- constrângerile (relații semantice între elementele de modelare, exprimate liber dar consistent);
- dependențele (relații de utilizare unidirecțională între două elemente);
- dualitățile (tip, instanță) și intențiile (tip, clasa).

1.2.3 TIPURI DE DIAGrame UML

Spre deosebire de standardul UML 1.x care propune 9 tipuri de diagrame⁴, standardul 2.x propune 14 astfel de tipuri, clasificate în 2 tipuri în funcție de informația oferită:

- Diagrame care oferă informații structurale (7). Acestea indică elementele sistemului modelat și sunt utilizate cu precădere în explicarea arhitecturii software a proiectului:

Diagrama de clasă⁵ (*class diagram*) descrie structura proiectului prezentând clasele acestuia (cu atributele specifice) și relațiile dintre acestea. Ea prezintă entități diferite și relațiile dintre acestea (structuri statice). Se pot afișa clase logice (în mediul business) sau clase de implementare (inginerie software - programare)⁶. Vizual⁷, acest tip de diagramă este reprezentat prin dreptunghiuri separate în 3 zone orizontale ce conțin numele clasei, atributele clasei și respectiv metodele clasei. Dreptunghiurile sunt unite prin săgeți ce reprezintă relațiile dintre acestea (de exemplu moștenirea se reprezintă printr-o linie cu o săgeată plină iar relațiile de asociere printr-o linie simplă). Exemplificare în figura de mai jos:

⁴ Diagrama cazurilor de utilizare, diagrama de clase, diagrama de obiecte, diagrama de stări, diagrama de secvență, diagrama de colaborare, diagrama de activități, diagrama de componente și diagrama de plasare.

⁵ O clasă reunește obiectele cu stare și comportament similar (cu anumite proprietăți). Pe baza acestora vor fi create obiectele.

⁶ În acest caz, diagrama claselor furnizează structura codului care va scris.

⁷ Pentru clasele de implementare.

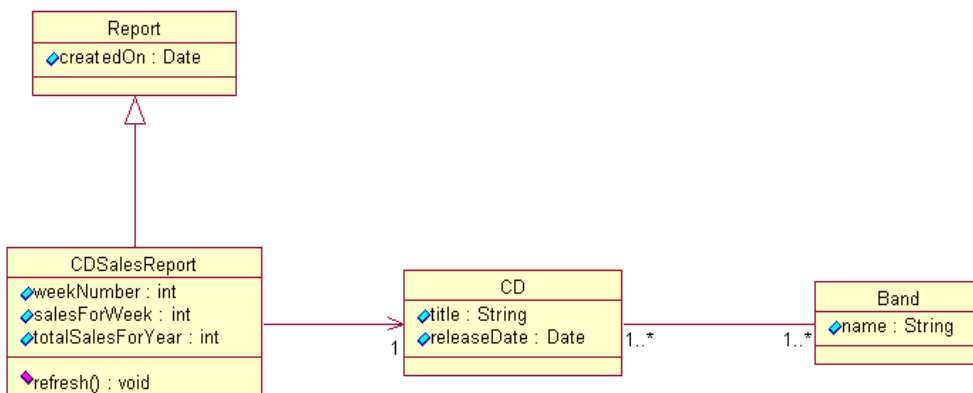


Diagrama de componente (*component diagram*) descrie modul în care proiectul software este împărțit în componente fizice și relațiile dintre aceste componente (dependințe). De obicei va prezenta dependența de alte componente software⁸, la diferite nivele (global sau pe anumite porțiuni semnificative). Exemplificare în figura de mai jos:

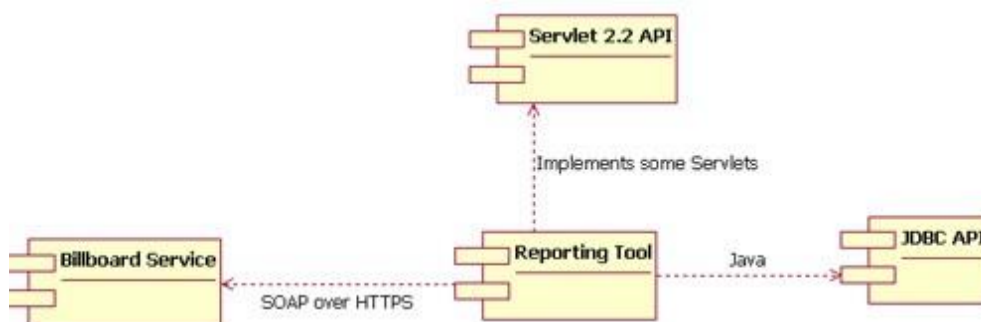


Diagrama structurii compozite (*composite structure diagram*) descrie structura internă a unei clase. Acest tip de diagramă poate avea la rândul ei componente interne, precum și porturi de acces prin care componentele interacționează între ele (prin intermediul conectorilor). O structură compozită reprezintă un set de elemente interconectate care colaborează la un moment dat spre atingerea unui scop. Exemplificare în figura de mai jos:

⁸ Cum ar fi bibliotecile externe.

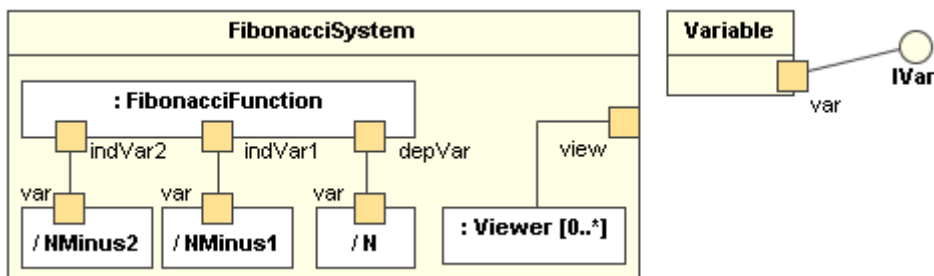


Diagrama de lansare (*deployment diagram*) descrie partea hardware și mediul de execuție în care va rula viitoarea aplicație și va prezenta modul în care diversele componente ale sistemului vor comunica între ele precum și locul în care vor rula (vis-a-vis de sistemul hardware). Notățiile vor fi similare diagramei de componente, dar va include și noduri (mașini virtuale sau fizice), reprezentate prin cuburi (3D) însoțite de nume. Ea oferă o vedere fizică asupra sistemului. Exemplificare în figura de mai jos:

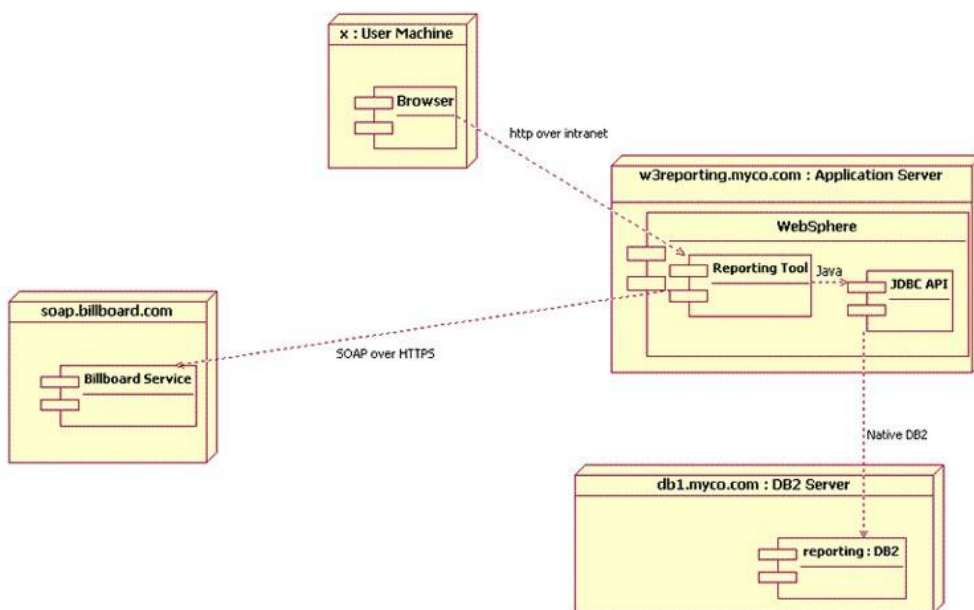


Diagrama de obiecte (*object diagram*) descrie o vedere (parțială sau totală) a structurii unei instanțe a sistemului modelat, la un moment dat (doar aspectele de interes sunt modelate de obicei). Un set de astfel de diagrame vor descrie modul în care sistemul poate evolua în timp. Spre deosebire de diagrama de clase, acest tip de diagramă poate prezenta exemple sau cazuri de

test pentru diagrama de clase⁹ și se folosește mai ales pentru tehnicile de *reverse engineering*. Exemplificare în figura de mai jos:

Object diagram of an order management system

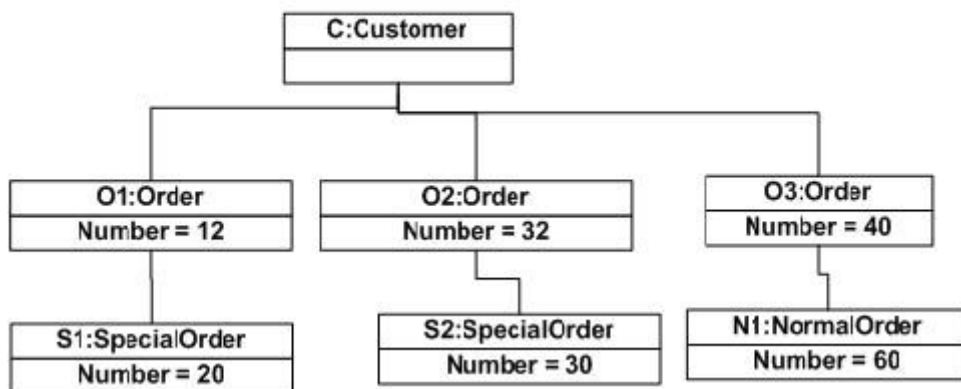


Diagrama de pachete (*package diagram*) descrie componentele logice ale unui proiect software și relațiile dintre acestea (dependințe). Aceste dependințe sunt specificate în standardul UML (sunt un set restrâns). Acest tip de diagramă poate fi folosit pentru a ilustra funcționalitatea sistemului și poate reprezenta sistemul pe diferite nivele. Pachetele pot conține subpachete (structură arborescentă) și pot grupa elementele UML după anumite criterii (de exemplu gruparea unei clase). Exemplificare în figura de mai jos:

⁹ Putem considera acest tip de diagramă ca un *snapshot* (la un moment dat) al unui sistem care rulează.

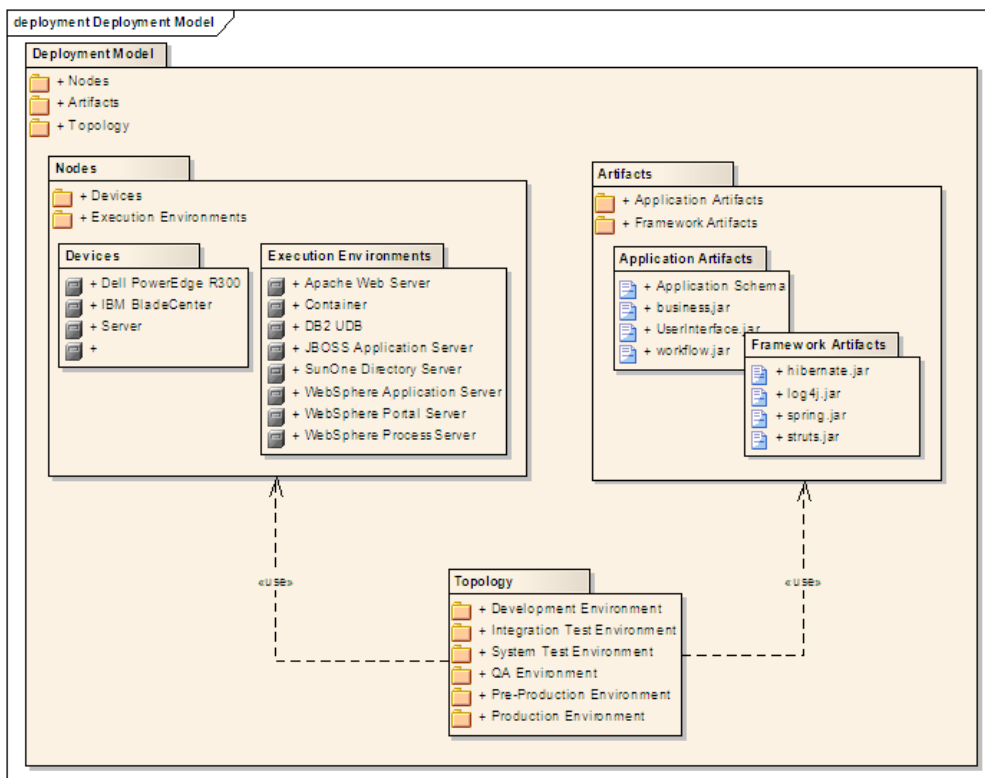
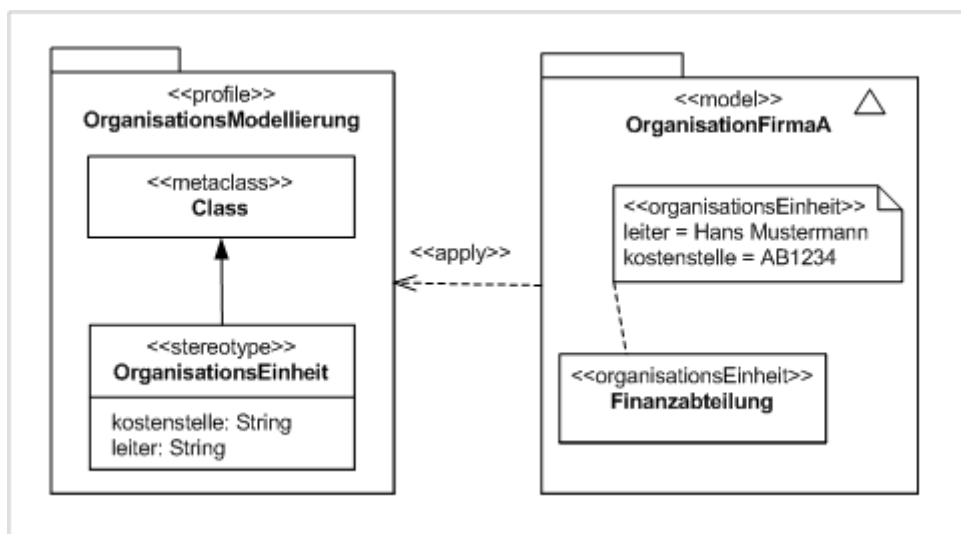


Diagrama de profil (metamodel) (profile diagram) descrie clasele (marcate cu `<<stereotype>>`) și pachetele (marcate cu `<<profile>>`) precum și relațiile dintre acestea (de exemplu extinderea claselor). Acest tip de diagramă este specifică standardului 2.x. Exemplificare în figura de mai jos:



- Diagrame care oferă informații comportamentale (7). Acestea indică ceea ce se întâmplă în sistemul modelat și sunt utilizate cu precădere în descrierea funcționalității proiectului software. Primele 3 tipuri de diagrame prezintă comportamente iar ultimele 4 prezintă interacțiuni (fluxuri de lucru și de date în cadrul sistemului):

Diagrama de activități (*activity diagram*) descrie fluxul de lucru operațional¹⁰ (de obicei la nivel global) al componentelor (claselor) proiectului software. O diagramă de stare are 5 elemente de bază: punctul de start inițial (notat cu un cerc plin) conectat la activitatea inițială, tranziții (notate cu săgeți), activitățile (notate printr-un pătrat cu colțuri rotunjite și care conține numele activității), puncte de decizie (cercuri sau romburi) care includ condiții pentru a permite tranziții și puncte de terminare (notate prin cercuri ce conțin un cerc plin în interior). Activitățile pot fi grupate în unități virtuale ce indică obiectul ce realizează activitatea. Exemplificare în figura de mai jos:

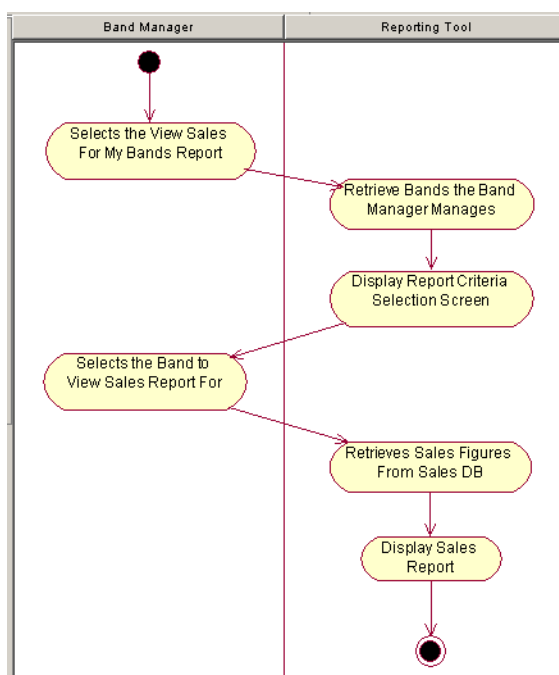


Diagrama de stări (*state diagram, statechart diagram*) descrie stările și tranzițiile de stare ale claselor sistemului (modelare dinamică). De obicei doar clasele cu mai multe stări posibile în timpul activității sistemului vor fi

¹⁰ Procesarea unei activități, de exemplu.

modelate. O diagramă de stare are 5 elemente de bază: punctul de start inițial (notat cu un cerc plin), tranziții între stări (notate cu săgeți) reprezentând relațiile dintre două stări, stările (notate printr-un pătrat cu colțuri rotunjite) – care pot conține acțiuni și activități executate de sistem în acea stare, puncte de decizie (cercuri) și puncte de terminare (notate prin cercuri ce conțin un cerc plin în interior). Exemplificare în figura de mai jos:

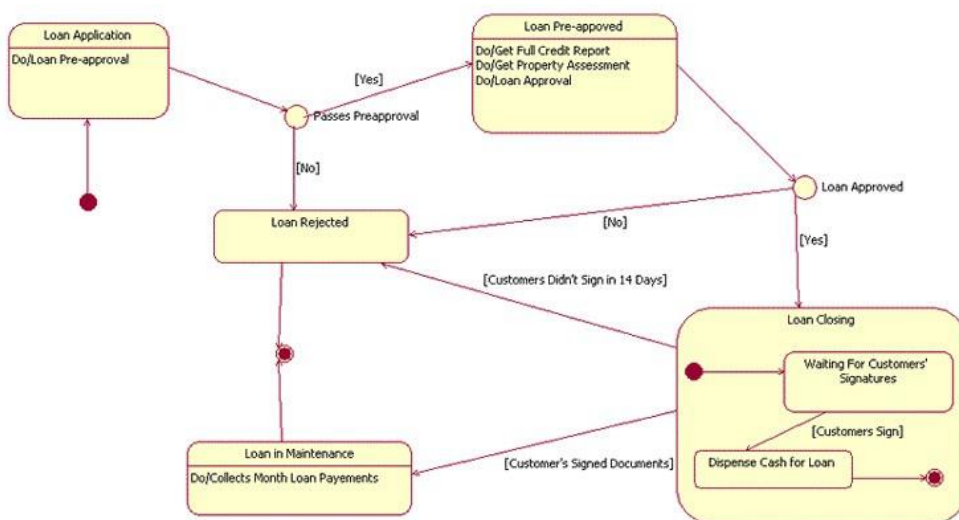


Diagrama de cazuri de utilizare (*use case diagram*) descrie funcționalitatea sistemului din punctul de vedere al unor actori (utilizatori) care au anumite scopuri - reprezentate de cazurile de utilizare (și dependențele dintre aceste cazuri). Rolul acestei diagrame este să ajute, în cadrul echipei de dezvoltare, la vizualizarea cerințelor funcționale ale sistemului și a legăturilor între actori (și procesele esențiale) și între diversele cazuri de utilizare. Diagrama poate cuprinde toate cazurile de utilizare posibile în cadrul sistemului sau doar un grup de cazuri înrudite funcțional (de exemplu modelarea cazurilor de utilizare legate de securitatea sistemului). Este important de reprezentat cazurile de utilizare în mod punctual, punând accentul pe scopul actorului în acel caz.

Cazurile de utilizare sunt modelate prin ovale centrale, iar actorii prin figuri stilizate reprezentând oameni. Relațiile dintre acestea sunt modelate prin linii, ca în figura de mai jos:

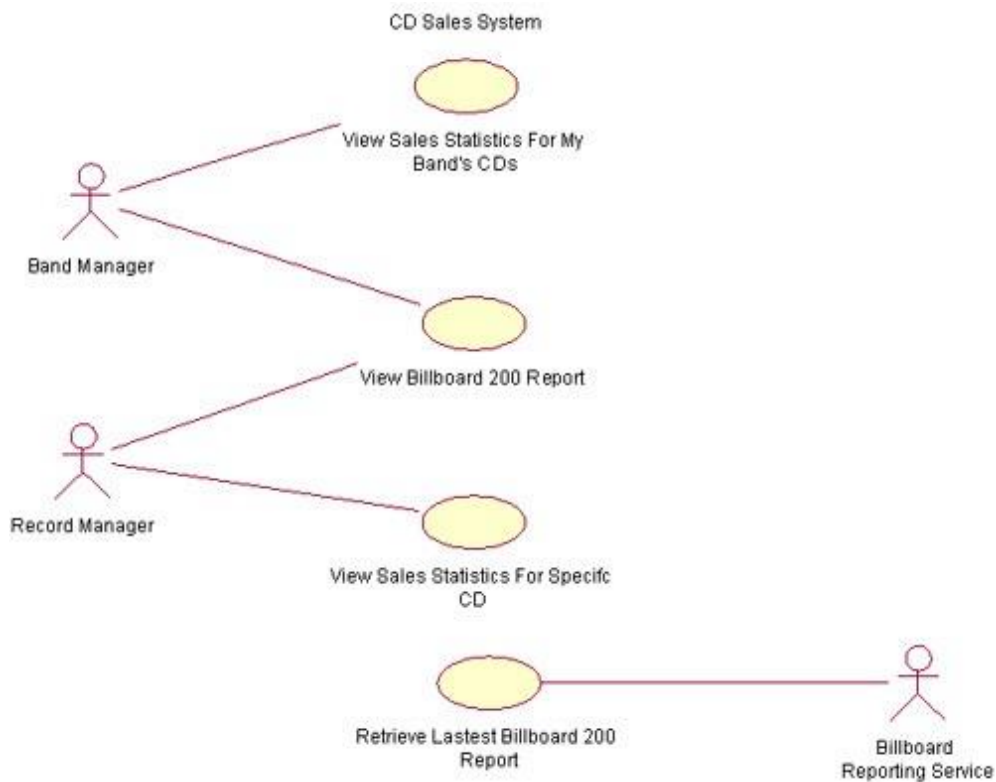


Diagrama de comunicare (*communication diagram*) descrie interacțiunea (statică și dinamică) dintre componente (prezintă informații din diagramele de clase, secvențe și cazuri de utilizare). Comunicația poate fi sincronă, asincronă sau de tip întoarcerea la o rutină ierarhic superioară (*return*). Exemplificare în figura de mai jos:

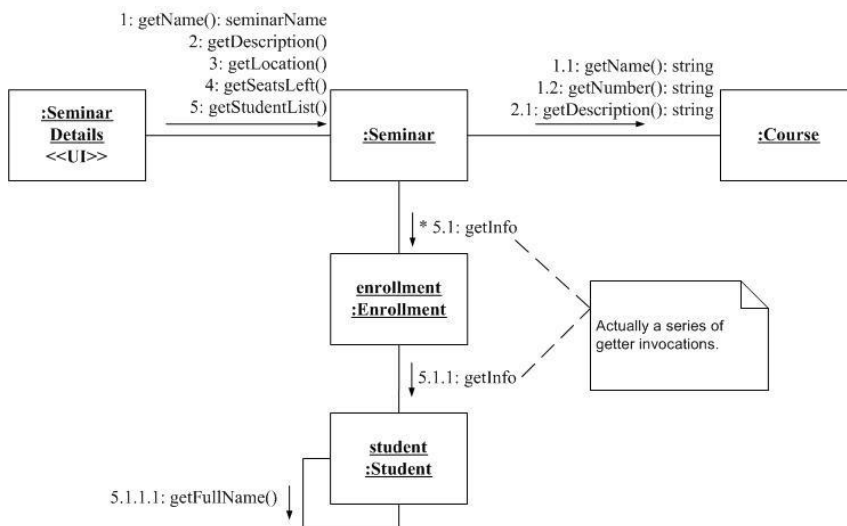


Diagrama de interacțiuni (*interaction overview diagram*) descrie arhitectura generală a sistemului modelat, în care fiecare nod reprezintă diagrame de comunicare. Exemplificare în figura mai jos:

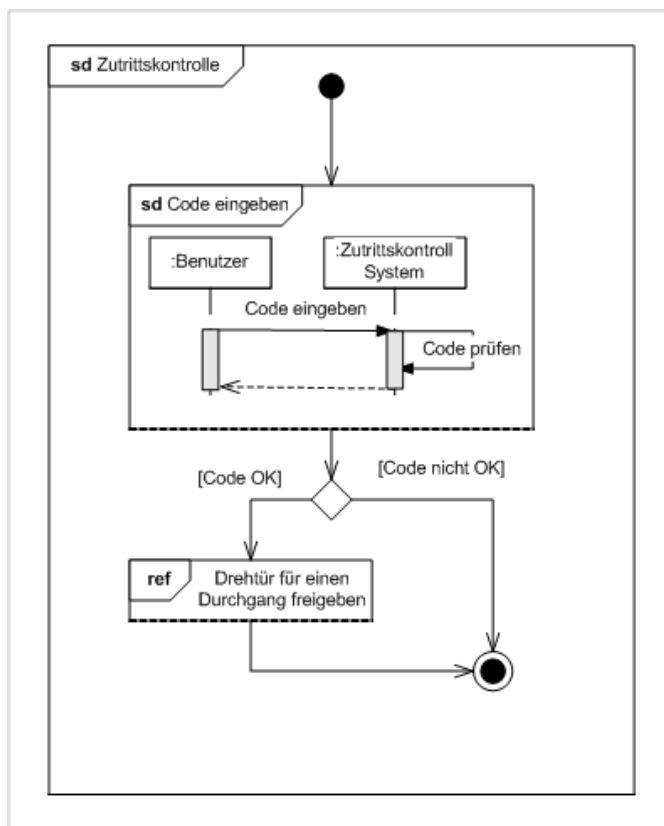


Diagrama de secvențe (*sequence diagram*) descrie modul în care obiectele comunică între ele prin mesaje¹¹, în ordinea în care aceste interacțiuni apar¹². Scopul ei este să definească secvențele de evenimente care rezultă într-o stare dezirabilă a sistemului (de exemplu se atinge un scop propus). Axa verticală reprezintă (de sus în jos) momentul de timp al mesajelor iar cea orizontală reprezintă (de la stânga la dreapta) obiectele care primesc-trimit acele mesaje. Acest lucru este exemplificat în figura de mai jos:

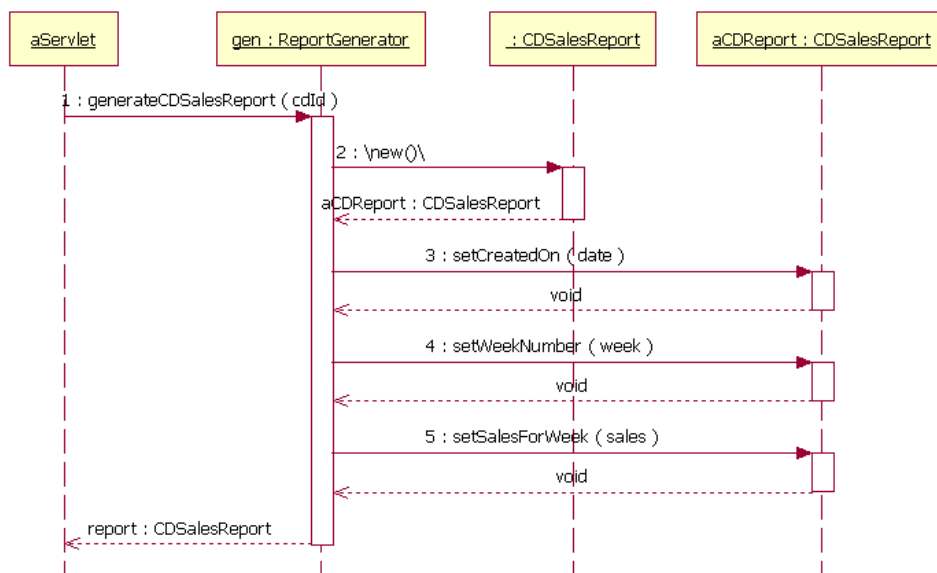
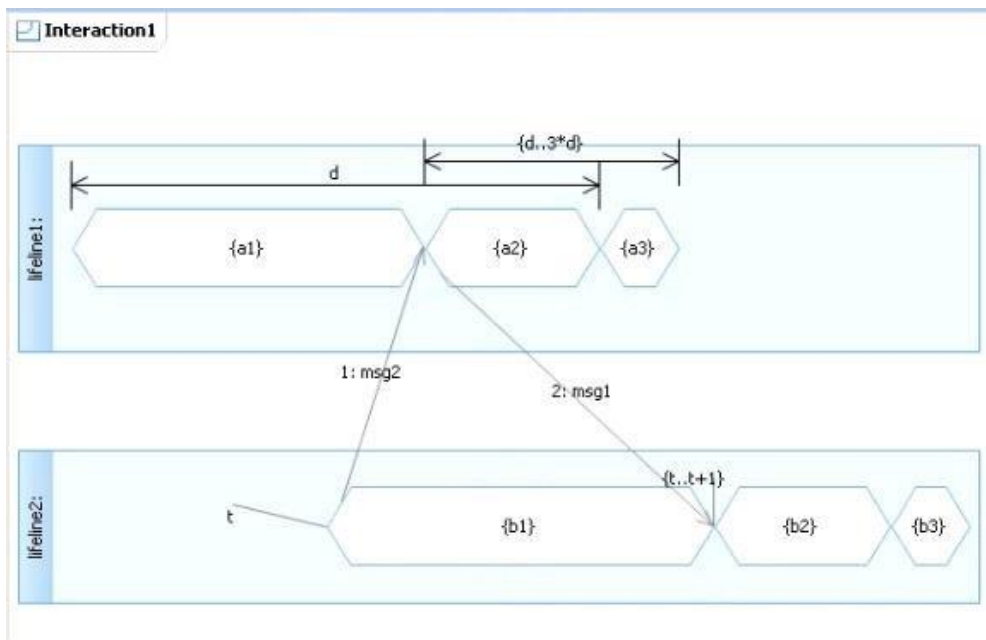


Diagrama de timpi (*timing diagram*) descrie constrângerile de temporizare prezente în sistem, în special constrângerile de timp sau durata mesajelor sau a condițiilor dinamice. Un proiect poate include mai multe asemenea diagrame (în funcție de setul de constrângeri ales, de cui se aplică acest set, de ce aspecte ale temporizării ne interesează, de duratele de viață, etc.), diagrama fiind de obicei o reprezentare parțială a proiectului. Elementele diagramei de timp se vor regăsi și în diagrama de secvențe (deși reciproca nu este întotdeauna adevărată). Diagrama de timpi poate conține stări sau condiții. Acest lucru este exemplificat în figura de mai jos:

¹¹ Un mesaj reprezintă un element al diagramei UML care definește un tip specific de comunicare în cazul unei interacțiuni.

¹² Accentul nu se pune pe mesajele în sine (deși este bine de precizat care sunt acestea), ci pe ordinea acestora.



1.3 APLICAȚII DE DESENARE ȘI MODELARE UML

1.3.1 GENERALITĂȚI

Diferența între modelele și diagramele UML este că, în cazul modelelor, acestea sunt reprezentări abstracte (la diferite nivele de abstractizare) ale unui sistem, ce conțin diagrame pentru instanțierea (și reprezentarea grafică a) unui anumit aspect al modelului; o diagramă reprezintă un aspect al unui model. Diagramele în sine ilustrează aspectele cuantificabile ale sistemului (într-un mod vizual).

Există unelte software care realizează modelări bazate pe limbajul UML a proiectelor software. De obicei acestea includ facilități pentru desenarea efectivă a diagramei, dar conțin și un motor de modelare care preia elementele individuale (descrise folosind noțiunile UML) și realizează un o

descriere completă¹³ a sistemului proiectat. Alte aplicații software se opresc la desenarea și generarea diagramelor UML, fără a modela complet sistemul.

1.3.2 COMPARAȚII

Unele din cele mai des folosite aplicații software în modelarea/proiectarea UML sunt descrise pe scurt în continuare:

Nume	Licență	Implement.	Facilități suportate	Detalii
	UML).		Generează diagrame din cod sursă (nativ Java, alte limbaje prin pluginuri).	software. http://argouml.tigris.org/
Paradigm for UML	(ediție gratuită – personală și comunitară; versiune academică disponibilă).	(crossplatform).	Generează cod din diagrame pentru limbajele: Java, C#, C++, PHP, Ada și Action Script (doar versiunea comercială). Generează diagrame din cod sursă (Java, C# - binary, C++, PHP; doar în versiunea comercială).	NetBeans. Implementează complet specificațiile UML 2.1. Versiunile gratuite au facilități reduse sau introduc watermark-uri în produsul final. http://www.visual-paradigm.com/product/vpuml/
IBM Rational Software Architect	Comercială, IBM EULA.	Multiplatform.	UML 1.x.	Nu mai este dezvoltat. Este înlocuit de IBM Rational Software Modeler (UML 2.0). http://www-01.ibm.com/software/rational/announce/rose/
BoUML	Gratuit până la v. 4.23 (GPL), comercial peste v. 5.00.	C++ / Qt (crossplatform).	UML 2.x. Generează cod din diagrame pentru limbajele: C++, Java, PHP, IDL, Python. Generează diagrame din cod sursă (C++, Java, PHP).	Foarte rapid comparativ cu alte modelatoare UML. Apreciat de comunitatea software care dorește o unealtă software performantă și „lightweight”. http://www.bouml.fr/
ArgoUML	Gratuit, EPL. Versiune comercială derivată (Poseidon for UML).	Java (crossplatform), Java Web Start.	UML 1.4. Generează cod din diagrame pentru limbajele: C++, C#, Java, PHP4, PHP5, Ruby. Generează diagrame din cod sursă (nativ Java, alte limbaje prin pluginuri).	Respectă fidel standardul UML 1.4. Implementare incompletă a standardului UML 1.4. Foarte apreciat de comunitatea software. http://argouml.tigris.org/

¹³ Completă bineînțeles dacă utilizatorul va descrie complet sistemul!

Nume	Licență	Implement.	Facilități suportate	Detalii
Umbrello UML Modeller	Gratuit, GPL.	C++ / KDE (Linux, Windows).	UML 1.x. Generează cod din diagrame pentru limbajele: ActionScript, Ada, C#, C++, D, IDL, Java, JavaScript, Pascal, Perl, PHP, Python, Ruby, SQL, Tcl, XML Schema . Generează diagrame din cod sursă (C++, IDL, Pascal/Delphi, Ada, Python, Java; import XML, RoseMDL).	Exportă cod sursă într-un număr mare de formate. Face parte din suita de software KDE (Linux). Permite scrierea facilă de generatoare de cod pentru diferite limbaje de programare. http://uml.sourceforge.net/
Software Ideas Modeller	Comercial (ediție gratuită disponibilă).	C# / .net, MONO (Windows, Linux).	UML 2.2. Generează cod din diagrame pentru limbajele: C++, C#, Java, PHP, Python, Ruby, SQL DDL, VB.NET, VB6. Generează diagrame din cod sursă (C#, VB.NET).	Permite generarea de diagrame mixte, de diagrame ale robusteții modelului (în total 16 noi diagrame). Suport UML 2.2 complet. http://www.softwareideas.net
Modelio	Gratuit, GPL 3 & Apache. Versiune comercială disponibilă.	Java, C++ (Windows, Linux).	UML 2.0. Generează cod din diagrame pentru limbajele: Java, C++, C#, XSD, WSDL și SQL. Generează diagrame din cod sursă (Java, C++, C#).	Modular. Extensibil. Suportă complet standardul UML 2.0. http://www.modelio.org/
yUML	Online (serviciu). Versiune comercială stand-alone.	-	Momentan suportă doar diagramele de clase și de cazuri de utilizare.	Relativ simplu, adecvat proiectelor mici. Ușor de folosit (complexitate scăzută). Limbajul UML este incomplet implementat. http://yuml.me/
UMLet	Gratuit, GPL.	Java (crossplatform).	-	Relativ simplu, adecvat proiectelor mici. Ușor de folosit (complexitate scăzută). Poate rula ca și plugin Eclipse. http://www.umlet.com/
StarUML	Gratuit, GPL modificat.	Delphi (Windows).	UML 2.0. Generează cod din diagrame pentru limbajele: Java, C# și C++. Generează diagrame din cod sursă (profile Java și C++, cod sursă C#).	Partea de modelare este incomplet dezvoltată (nu generează toate diagramele specificate de standardul OMG). Arhitectură modulară, MDA, suportă pluginuri (limbaje multiple de programare), ușor de folosit. Nu mai este dezvoltat (din 2005). http://staruml.sourceforge.net/en/

Nume	Licență	Implement.	Facilități suportate	Detalii
plantUML	Gratuit, GPL.	Java (crossplatform).	UML 1.x.	Partea de modelare incomplet dezvoltată, nu face verificări interne. Ușor de folosit. http://plantuml.sourceforge.net/
yEd	Gratuit.	Java (crossplatform).	-	Algoritm de poziționare automată a layout-ului. http://www.yworks.com/en/products_yed_about.html
Violet UML Editor	Gratuit, GPL.	Java (crossplatform).	UML 1.x. Nu suportă generarea de cod sau <i>reverse engineering</i> .	Relativ simplu. Ușor de folosit (complexitate scăzută). Rulează ca și applet Java sau ca plugin Eclipse. http://alexdp.free.fr/violetumleditor/page.php
Eclipse MDT UML2 Tools, diverse pluginuri	Gratuit, EPL.	Java (crossplatform).	UML 2.x. Generează cod din diagrame pentru limbajele: Java. Generează diagrame din cod sursă (Java).	Necesită platforma Eclipse. Suportă 5 tipuri de diagrame (specificații incomplet implementate). http://www.eclipse.org/modeling/mdt/?project=uml2
NetBeans (plugin), JDeveloper	Gratuit, CDDL.	Java (crossplatform).	Jdeveloper oferă o soluție de modelare completă.	Disponibil în NetBeans 6.7 Versiunile mai noi folosesc pluginuri corespunzătoare (suportă plugin Visual Paradigm). http://netbeans.org/projects/uml/
MS Visio	Comercial.	-	UML 2.2.	Partea de modelare este puțin dezvoltată, fiind folosit mai mult pentru crearea de diagrame. http://visio.microsoft.com/en-au/pages/default.aspx
Dia (cu Dia2Code)	Gratuit, GPL.	C/GTK+ (crossplatform), cu extensii Python.	UML 1, parțial UML 2. Generează cod din diagrame pentru limbajele: Ada, C, C++, Java, PHP, PHP5, Python, Ruby, SQL și C#. Nu generează diagrame din cod sursă.	Partea de modelare este puțin dezvoltată, fiind folosit mai mult pentru crearea de diagrame. http://live.gnome.org/Dia http://dia2code.sourceforge.net/
MS Visual Studio 2010	Comercial.	C++, .NET (Windows).	Generează cod din diagrame pentru limbajele: C# și VB.NET. Generează diagrame din cod sursă (C# și VB.NET). Validează cod C/C++ comparativ cu diagrama de nivel (controlul arhitecturii).	Partea de modelare este incomplet dezvoltată (nu generează toate diagramele specificate de standardul OMG - doar suport de bază). http://www.microsoft.com/visualstudio/en-us

1.3.3 VISUAL PARADIGM FOR UML

Pentru familiarizarea cu setul de controale și facilitățile oferite de Visual Paradigm precum și cu modul de creare a modelelor și al diagramelor UML, puteți utiliza și următoarele tutoriale pentru modelarea UML:

<http://www.visual-paradigm.com/product/ag/provides/umlmodeling.jsp#usecasediagram>

http://www.visual-paradigm.com/support/documents/vpumluserguide/94/95_usecasemodel.html

http://www.visual-paradigm.com/support/documents/vpumluserguide/12/24_gettingstart.html

<http://www.youtube.com/user/VisualParadigm>

Un tutorial video pentru utilizarea diagramelor UML și a aplicației Visual Paradigm for UML se găsește la adresa:

<http://www.viswiser.com/training/usingusecasediagram.html>

Accesarea tutorialului necesita înscrierea (gratuită) pe portalul visWiser:

http://www.viswiser.com/free_subscription.jsp

Obs: nu toate tutorialele oferite prin portal pot fi vizionate în mod gratuit!

1.3.4 ALTE APLICAȚII

Tutoriale pentru utilizarea altor aplicații de modelare-creare diagrame UML sunt date mai jos:

MS Visio: <http://www.visio-tutorial.com/>

BoUML: <http://bouml.free.fr/tutorial/tutorial.html>

StarUML: [http://staruml.sourceforge.net/docs/user-guide\(en\)/toc.html](http://staruml.sourceforge.net/docs/user-guide(en)/toc.html)

[http://staruml.sourceforge.net/docs/user-guide\(en\)/ch04.html](http://staruml.sourceforge.net/docs/user-guide(en)/ch04.html)

<http://www.liacs.nl/~chaudron/se2010/A%20Brief%20Guide%20on%20UML%20Modeling%20with%20StarUML.pdf>

ArgoUML: <http://argouml.tigris.org/files/documents/4/0/argouml-0.16/quickguide-0.16.pdf>

Umbrello	UML	Modeller:
http://docs.kde.org/stable/en/kdesdk/umbrello/umbrello.pdf		

http://lib.custis.ru/images/c/c4/PlantUML_Language_Reference_Guide.pdf

Software Ideas Modeller: <http://www.softwareideas.net/SimUserGuide.pdf>